

See, Plan, Rewind: Progress-Aware Vision-Language-Action Models for Robust Robotic Manipulation

Tingjun Dai*, Mingfei Han*[†], Tingwen Du, Zhiheng Liu, Zihao Zhang, Zhihui Li, Salman Khan, Jun Yu, Xiaojun Chang, *Senior Member, IEEE*

Abstract—Measurement of task progress through explicit, actionable milestones is critical for robust robotic manipulation. This progress awareness enables a model to ground its current task status, anticipate verifiable intermediate states, and detect and recover from failures when progress stalls. To embody this capability, we introduce **See, Plan, Rewind (SPR)**, a progress-aware vision-language-action framework that dynamically grounds language instructions into a sequence of spatial subgoals. SPR operates through a continuous core cycle, Seeing the current state and upcoming milestone, Planning a trajectory towards the next 2D waypoint, and Rewinding to a recoverable state upon failure by monitoring progress against the expected sequence. This closed-loop approach enables robust error correction without requiring additional training data or auxiliary models. Extensive experiments demonstrate the framework’s effectiveness, generalization and robustness: SPR outperforms the MolmoAct baseline by 5% on the LIBERO benchmark. On the challenging LIBERO-Plus benchmark with unseen instructions and initial states, SPR achieves state-of-the-art robustness with the smallest performance drop, surpassing OpenVLA-OFT and UniVLA, demonstrating superior out-of-distribution robustness.

Index Terms—Progress Awareness, Robotic Manipulation, Vision-Language-Action Models, Spatial Reasoning

1 INTRODUCTION

Robotic manipulation requires a continual, closed-loop interaction with a dynamic 3D environment. While existing approaches [1], [2], [3], [4], [5], [6], [7], [8] have enabled basic task execution and flexible behaviors, robust performance demands an agent to not only perceive and act, but also maintain a grounded, quantitative awareness of its progress toward a goal. We formalize this capability as *progress awareness*, the ability to measure task execution against a sequence of concrete and actionable milestones.

Recent research recognized this need and explored progress monitoring capabilities. Several approaches [9], [10], [11] have explored progress monitoring, such as ECOT [9] which connects semantics to features via visual chain-of-thought, yet its progress signals remain abstract and lack spatial grounding. In failure recovery, approaches include

*Authors contribute equally. [†]Project lead.

- Tingjun Dai, Mingfei Han, Tingwen Du, Zhihui Li and Xiaojun Chang are with School of Information Science and Technology, University of Science and Technology of China, Anhui, China. E-mail: {hmf282@gmail.com, dutw2023@mail.ustc.edu.cn, {lizhihuics,xjchang}@ustc.edu.cn}
- Tingjun Dai is also with University of Technology Sydney, Ultimo, NSW, Australia. E-mail: {tingjun.dai@student.uts.edu.au}
- Mingfei Han, Salman Khan are with Department of Computer Vision, Mohamed Bin Zayed University of Artificial Intelligence, Abu Dhabi, United Arab Emirates. E-mail: {mingfei.han, salman.khan}@mbzuai.ac.ae.
- Zhiheng Liu is with The University of Hong Kong, Hong Kong, China. E-mail: {zhihengl0528@connect.hku.hk}.
- Zihao Zhang is with Institute of AI for Industry, Chinese Academy of Sciences, Jiangsu, China. E-mail: {zhangzihao@ict.ac.cn}.
- Jun Yu is with School of Intelligent Science and Engineering, Harbin Institute of Technology (Shenzhen), Guangdong, China. E-mail: {yujun@hit.edu.cn}.

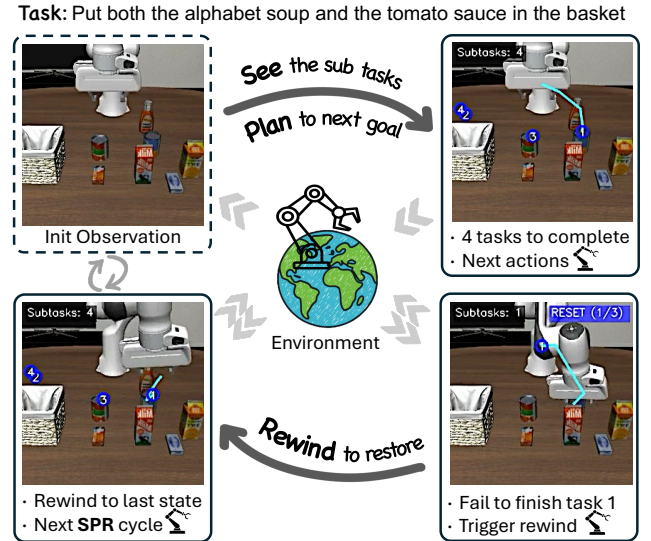


Fig. 1: *See-Plan-Rewind* framework’s closed-loop execution workflow. Starting from the initial state (top-left), the model performs See-Plan reasoning to generate actions (arrow to top-right), which visualizes the subtask decomposition and trajectory planning. Under normal execution, the loop returns to top-left. When progress monitoring detects failures (bottom-right), the Rewind mechanism activates (bottom-left), returning the robot to the initial state before resuming the loop. This closed-loop design enables autonomous error recovery through explicit progress awareness.

generating scalable failure-reasoning data [4], [5], [12] and

employing LLMs for error analysis and correction [6], [7]. However, methods like FailSafe [12] rely on extensive additional failure data collection, which is costly, while RE-FLECT [7] depends heavily on pre-defined LLM prompting, limiting adaptability in unseen scenarios. While these approaches advance progress awareness, their signals often remain abstract linguistic constructs or binary flags, lacking explicit spatial grounding for robot action. Also, their recovery mechanisms typically depend on auxiliary models or substantial data collection. This gap underscores the need for a unified framework that provides robot-interpretable, spatially-grounded progress planning with an intrinsically embedded and data-efficient recovery mechanism.

To address these limitations, we introduce **See, Plan, Rewind (SPR)**, a framework that endows vision-language-action models with explicit completion awareness of task milestones through concrete spatial subgoals. Unlike abstract descriptions and plans lacking measurable progress benchmarks, SPR leverages gripper interactions from existing demonstrations to decompose tasks into sequences of grounded spatial waypoints. Each waypoint serves as a verifiable intermediate target that provides perceptual anchoring, simplifies trajectory planning, and enables unambiguous progress evaluation. Specifically, SPR first *Sees* to identify remaining subtasks, then *Plans* 2D trajectories to the next sub-goal bounded waypoint, and finally *Rewinds* to restore in-distribution states upon detecting anomalies, without additional training data or auxiliary models.

To establish the explicit awareness, we propose an extensible pipeline that automatically decomposes tasks into spatially-grounded subgoals from demonstrations. For pick-and-place tasks, subtask boundaries are identified directly from gripper state transitions; for other manipulation types, we employ Gemini-3 to annotate subtask segments with boundary frames and semantic descriptions. For each segment, we extract 2D subgoal coordinates by leveraging DINOv3 [13] for gripper feature matching and SAM [14] for precise segmentation. Finally, the model learns to associate observations with spatial subgoals, task status, and motion trajectories within structured behavioral sequences. During execution, progress is monitored in a closed loop via a state recorder that continuously tracks predicted subtask counts and planned 2D trajectories. Sustained count increases indicate execution failures such as repeated failed grasps, while unchanged trajectories over extended timesteps signal progress stagnation where the robot remains trapped in unproductive states due to collisions, misalignment, or other environmental constraints. Both anomaly types trigger the Rewind mechanism to restore operational stability.

We evaluate SPR on both simulation and real-robot settings. On the LIBERO [15] benchmark, SPR surpasses MolmoAct baseline by 3.8%, and in the one-policy-for-all setup achieves an additional 1.2% improvement, in contrast to methods like OpenVLA-OFT [16] which regress in this challenging scenario. On the out-of-distribution LIBERO-Plus suite [17] with over 6800 test-time variants, SPR maintains minimal performance drop across variation types (18.8% average v.s. 27.0% for OpenVLA-OFT [16] and 37.5% for UniVLA [18]), establishing new state-of-the-art cross-domain robustness. We further validate SPR on three real-robot tasks, including one basic pick-and-place task and two

challenging scenarios involving long-horizon multi-object tidying and continuous-contact pushing, where the baseline fails entirely on both challenging tasks while SPR maintains consistent performance.

To summarize, we make the following contributions:

- **Progress Awareness with Spatial Subtasks:** We establish a new paradigm for progress monitoring by decomposing tasks into a sequence of 2D spatial subgoals, replacing abstract plans with concrete, verifiable waypoints that enable fine-grained, robot-executable progress tracking without auxiliary models.
- **Progress-Driven Error Recovery:** We formulate progress monitoring as an executable recovery policy that detects anomalies through progress tracking and restores the robot to in-distribution states.
- **Effectiveness and OOD Robustness:** We demonstrate that SPR achieves superior performance and generalization, outperforming strong baselines on the LIBERO benchmark and setting a new state-of-the-art for out-of-distribution robustness on LIBERO-Plus suite with minimal performance degradation.

2 RELATED WORKS

Vision-Language-Action Models. Building on the capabilities of pretrained vision foundation models and large language models, vision-language models (VLMs) [19], [20], [21], [22], [23] have demonstrated strong performance in multimodal understanding. Motivated by these strengths, researchers have begun treating robot control as an additional output modality, fine-tuning VLMs on large-scale robot datasets [24], [25], [26], [27], [28] to transfer their end-to-end prediction abilities to robotic tasks. Pioneering work RT-2 [29] co-fine-tunes on web-scale VQA and robot data, enhancing generalization and emergent capabilities. OpenVLA [30] provides an open-source counterpart that explores efficient fine-tuning via LoRA and quantization for deployment. The π_0 model [31] leverages high-quality data to improve performance and employs a flow-matching architecture to enhance real-time control capabilities. $\pi_{0.5}$ [32] generalizes to long-horizon household tasks in unseen environments through co-training on multi-modal data and decomposes tasks into semantic subtasks for hierarchical decision-making. Errors in manipulation can accumulate over time, making them susceptible to propagation. Therefore, perceiving execution progress and enacting timely corrections is vital for robust robot policies.

Progress Awareness and Execution Monitoring. Early progress monitoring relied on external human supervision, with systems like OLAF [1] using verbal guidance to synthesize recovery data and YAY [2] updating policies through direct interventions, both facing scalability limitations. Recent works have explored autonomous progress monitoring using VLMs [9], [10], [11], [33] through various mechanisms. ECOT [9] introduced visual chain-of-thought for subgoal decomposition and progress awareness. Other structured approaches include MolmoAct [10] generating mid-level spatial plans with sparse waypoints (though coarse sampling may limit precision) and SeqVLA [11] using a detection head for subtask completion awareness. However,

many VLM-driven approaches still depend on prompt-based heuristics and lack explicit grounding in dynamic physical task states, leading to unreliable progress awareness under visual ambiguity or occlusion.

Failure Recovery and Robust Control. While these monitoring capabilities enhance task awareness, the occurrence of failures during execution remains inevitable, making autonomous recovery crucial for preventing error accumulation and ensuring overall task success. Early failure recovery methods relied on external human knowledge [1], [2], [34], facing scalability limitations. With the emergence of VLMs, research shifted toward autonomous reasoning [6], [7], [35], [36], [37], where REFLECT [7] uses LLMs to interpret execution experiences and suggest corrections, while COME-robot [6] employs GPT-4V for adaptive replanning. Relying solely on VLMs’ reasoning often leads to unreliable recovery in novel environments, prompting methods like AHA [4], RoboFAC [5], and FailSafe [12] to generate targeted failure-and-recovery data. However, these data-driven methods require extensive failure data collection, which is costly. This motivates alternatives that leverage successful demonstrations to synthesize recovery behaviors, reducing dependency on failure-specific data.

3 SEE-PLAN-REWIND FRAMEWORK

We introduce our See-Plan-Rewind framework, a progress-aware vision-language-action model that achieves robust manipulation through fine-grained spatial subtask planning and error recovery. As illustrated in Figure 2, our approach operates through a continuous cycle: the model first Sees the current state and subtask remain (Section 3.1), Plans a trajectory towards next 2D waypoint, and Rewinds to a recoverable state when anomalies are detected (Section 3.3). The foundation of this framework lies in our comprehensive data generation pipeline (Section 3.2), which automatically constructs supervision for subtask boundaries, spatial coordinates, and rewind trajectories without additional human annotation or auxiliary models.

3.1 Fine-Grained Spatial Subtask Planning

Grasping and manipulating objects is so routine for humans that we rarely notice the sophisticated progress awareness driving these actions. Our brain naturally decomposes manipulation goals into intermediate milestones, plans hand trajectories toward each subgoal, and continuously monitors execution progress—all without conscious thought. Inspired by this cognitive process, we design a fine-grained spatial subtask planning mechanism that endows VLA models with analogous progress-aware reasoning. By explicitly modeling the “See-Plan” cycle—identifying remaining subtasks and planning trajectories to concrete spatial waypoints—our framework enables robots to approach manipulation with the structured, milestone-driven strategy that humans employ effortlessly.

3.1.1 Spatial Subtask See and Plan

As illustrated in Figure 2, upon receiving the observation and task instruction, the model performs the See phase: outputting depth perception tokens (following MolmoAct,

not shown in figure), predicting remaining subtask count, and sequentially generating semantic descriptions with 2D coordinates for each subtask, establishes comprehensive progress awareness. In the subsequent Plan phase, the model generates a 2D trajectory with up to 5 waypoints from the current gripper position to the next subtask goal. Unlike the baseline that plans directly toward the final goal, our next-subtask planning provides more robust execution guidance, particularly for long-horizon tasks where the final goal may be spatially irrelevant or even misleading before intermediate subtasks are completed, rendering end-to-end trajectory planning ineffective.

3.1.2 Action Reasoning with Subtask Awareness

Building upon MolmoAct’s depth-conditioned reasoning, our framework extends the autoregressive generation process to incorporate explicit subtask planning. Given an RGB observation I and language instruction T (which includes the action CoT prompt), the model sequentially generates five token streams: (i) depth perception tokens d ; (ii) remaining subtask count n ; (iii) subtask specifications $s = (s_1, \dots, s_n)$, where each $s_i = (\text{sem}_i, \text{coord}_i)$ consists of semantic description and 2D completion coordinates; (iv) trajectory tokens τ from current gripper position to the next subtask waypoint; (v) action tokens $a = (a_1, \dots, a_D)$ with D degrees of freedom. This factorization follows:

$$p(a, d, n, s, \tau | I, T) = p(d) \cdot p(n) \cdot p(s) \cdot p(\tau) \cdot p(a) \quad (1)$$

where each component conditions on the observation I , instruction T , and all previously generated tokens in the autoregressive sequence.

3.2 See-Plan Data Curation

Our See-Plan framework requires two types of supervision: (i) subtask boundaries and counts, (ii) spatial coordinates for subtask waypoints and trajectories. We design an automated pipeline to extract all supervision signals from existing demonstration data, eliminating the need for additional human annotation or auxiliary vision-language models during both training and inference.

3.2.1 Subtasks Segmentation and Description.

Our method segments demonstration trajectories into meaningful subtasks through task-adaptive boundary detection. For pick-and-place tasks, we identify subtask boundaries directly from gripper state transitions (open/close), which reliably mark subtask completion points. For tasks where gripper actuation alone is insufficient to delineate subtasks (e.g., pushing, closing cabinets), we employ Gemini-3 to generate subtask annotations, including start and end frame indices and semantic descriptions for each segment. For each frame, we compute the remaining number of subtasks, which serves as ground truth for training the model’s progress monitoring. As a special case, since LIBERO [15] tasks are predominantly pick-and-place, we apply the gripper-based segmentation and prompt DeepSeek-R1 [38] with the task instruction and detected subtask count to generate semantic descriptions for each segment.

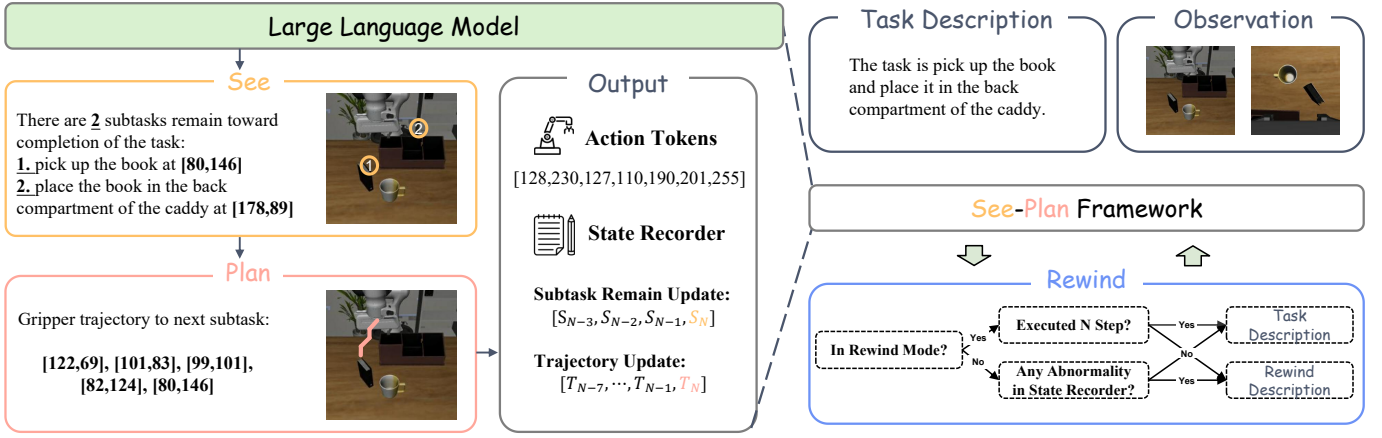


Fig. 2: SPR framework overview. *Left:* Upon receiving the task description and observation, the model performs See-Plan reasoning (Sec. 3.1), which identifies remaining subtasks with 2D spatial coordinates (See) and plans a gripper trajectory to the next subtask waypoint (Plan), then outputs action count tokens for execution. Each inference step also updates the state recorder, where S_N denotes the predicted subtask count and T_N the planned 2D trajectory at the current timestep. *Right:* The Rewind mechanism (Sec. 3.3) examines the state recorder after each step: if no anomaly is detected, the original task description is retained; if sustained anomalies are identified, the task description is switched to a rewind instruction for N steps before reverting to normal execution. Data generation in Sec. 3.2.

3.2.2 Gripper Trajectory Extraction.

To obtain spatial coordinates for subtask waypoints and 2D trajectories, we combine DINOv3 [13] and SAM [14] for robust gripper detection without task-specific training. We maintain a reference image of the gripper endpoint and use DINOv3’s patch-level features to identify the image region with highest similarity. This coarse localization is then refined by SAM, using the previously obtained position as point prompt for gripper segmentation. Following, we compute final coordinates by leveraging each method’s strength: x from SAM’s bounding box center (precise horizontal boundaries) and y from DINOv3 detection (accurate vertical endpoint localization). All 2D coordinates are discretized to $[0, 255]$. Finally, we smooth the trajectory by detecting and interpolating over outlier points with unusually large movements, and then applying a median filter with a small temporal window while preserving subtask boundaries. Given gripper coordinates for all frames, we extract subtask waypoints as the gripper positions at detected boundary frames. We uniformly sample 1-5 gripper positions from the current frame to the next subtask completion frame, providing intermediate waypoints for trajectory planning.

3.3 Error Recovery via Progress-Aware Rewind

While our subtask planning framework enables accurate progress monitoring, robust spatial planning alone cannot eliminate all failure modes. Inspired by human problem-solving strategies of “stepping back” when encountering obstacles, we propose a Rewind mechanism that disengages the model from erroneous states through a brief learned retraction, preventing the robot from persisting in failure modes with diminishing returns. Leveraging our framework’s subtask counting and 2D trajectory grounding capabilities, we can detect potential execution anomalies in real time and trigger timely recovery—implemented entirely

through joint training on constructed rewind data, without additional recovery demonstrations or auxiliary models.

3.3.1 Rewind Data Construction

To endow the model with rewind capability, we construct reverse trajectories by inverting successful forward demonstrations from the first subtask waypoint back to the robot’s initial position. The construction involves: (i) temporal reversal of frame sequences, (ii) negation of action token values (inverting end-effector delta movements), and (iii) setting the task instruction to “return to initial position.” All other supervision signals—subtask boundaries, waypoint coordinates, and 2D trajectories—are automatically inherited from the data curation pipeline described in Section 3.2.

3.3.2 Progress-Based Anomaly Detection

We maintain a state recorder, a first-in-first-out queue that continuously tracks the model’s predicted subtask counts over the most recent 4 timesteps and the planned 2D trajectories over the most recent 8 timesteps. Anomalies are detected through two complementary criteria:

Subtask Count Anomaly. Under normal execution, the predicted subtask count either remains constant or decreases monotonically as subtasks complete. We detect anomalies by examining whether the subtask count increases across both the current and the immediately prior window in the queue, indicating sustained execution failure that causes the model to regress to earlier stages.

Progress Stagnation. If the planned 2D trajectories remain identical across all 8 recorded timesteps, we identify the robot as trapped in an out-of-distribution state caused by collisions, misalignment, or other unexpected environmental conditions. In such states, the model encounters configurations absent from its training data and fails to generate effective actions, resulting in repeated identical plans without meaningful progress toward the current subtask.

TABLE 1: Out-of-distribution robustness evaluation on LIBERO-Plus benchmark. We report task success rates across five perturbation types, with subscripts indicating performance degradation relative to the original LIBERO test sets. **Bold** and underlined values denote the best and second-best success rates, respectively. **Red bold** and red underlined subscripts highlight the smallest and second-smallest performance drops, indicating superior OOD robustness. Our method achieves the highest average success rate (71.8%) with the smallest average degradation (18.8%), demonstrating strong generalization to unseen perturbations.

Method	LIBERO-PLUS					
	Background	Robot	Language	Layout	Light	Avg
OpenVLA [30]	25.3% _{↓51.2%}	4.1% _{↓72.4%}	26.8% _{↓49.7%}	31.6% _{↓44.9%}	4.4% _{↓72.1%}	18.7% _{↓57.8%}
OpenVLA-OFT [16]	83.6% _{↓14.0%}	30.6% _{↓67.0%}	83.6% _{↓14.0%}	73.2% _{↓24.4%}	91.6% _{↓6.0%}	<u>70.6%</u> _{↓27.0%}
OpenVLA-OFT_w [16]	92.5% _{↓2.8%}	43.7% _{↓51.6%}	73.2% _{↓22.1%}	72.3% _{↓23.0%}	68.2% _{↓27.1%}	68.5% _{↓26.8%}
π_0 [31]	78.5% _{↓15.7%}	6.6% _{↓87.6%}	61.0% _{↓33.2%}	70.4% _{↓23.8%}	79.6% _{↓14.6%}	56.6% _{↓37.6%}
π_0 -fast [39]	67.7% _{↓17.8%}	24.8% _{↓60.7%}	63.3% _{↓22.2%}	70.3% _{↓15.2%}	73.0% _{↓12.5%}	58.4% _{↓27.1%}
WorldVLA [40]	14.5% _{↓64.6%}	30.2% _{↓48.9%}	44.2% _{↓34.9%}	39.4% _{↓39.7%}	29.4% _{↓49.7%}	32.8% _{↓46.3%}
Nora [41]	50.5% _{↓37.4%}	41.1% _{↓46.8%}	67.0% _{↓20.9%}	63.9% _{↓24.0%}	31.0% _{↓56.9%}	51.8% _{↓36.1%}
UniVLA [18]	80.0% _{↓15.2%}	50.3% _{↓44.9%}	71.8% _{↓23.4%}	34.3% _{↓60.9%}	59.1% _{↓36.1%}	57.7% _{↓37.5%}
Ours	<u>86.0%</u> _{↓4.6%}	<u>47.7%</u> _{↓42.9%}	<u>78.5%</u> _{↓12.1%}	69.6% _{↓21.0%}	<u>85.0%</u> _{↓5.6%}	71.8% _{↓18.8%}

TABLE 2: Performance on LIBERO benchmark. We report results for two training configurations: separately-trained models (Ours) fine-tuned individually on each subset, and a jointly-trained model (Ours*) trained on all four subsets. **Bold** and underlined values indicate the best and second-best results respectively.

Method	LIBERO				
	Spatial	Object	Goal	Long	Avg
Diffusion Policy [42]	78.3%	92.5%	68.3%	50.5%	72.4%
Octo [43]	78.9%	85.7%	84.6%	51.1%	75.1%
OpenVLA [30]	84.7%	88.4%	79.2%	53.7%	76.5%
GRAPE [8]	88.5%	92.1%	83.1%	57.2%	80.2%
ThinkAct [3]	88.3%	91.4%	87.1%	70.9%	84.4%
π_0 -fast [39]	96.4%	96.8%	88.6%	60.2%	85.5%
MolmoAct [10]	87.0%	<u>95.4%</u>	87.6%	77.2%	86.8%
Ours	92.4%	93.0%	94.2%	<u>82.8%</u>	<u>90.6%</u>
Ours*	<u>93.2%</u>	<u>95.4%</u>	<u>93.2%</u>	85.4%	91.8%

Both criteria require anomalies to persist over multiple timesteps, filtering transient prediction noise while reliably identifying sustained execution failures.

3.3.3 Rewind Execution Strategy

Through joint training on forward demonstrations and constructed rewind data, the model learns to retreat to its initial position upon receiving a “return to initial position” instruction. During execution, if an anomaly is detected, we substitute the original task instruction with this rewind command for a fixed duration of N timesteps. This allows the robot to backtrack toward its starting configuration. After N steps, the system reverts to the original instruction and resumes the task from the new state. The duration N is critical: a value too small fails to provide sufficient operational clearance, while a value too large risks the arm exiting the camera’s view or assuming unrecoverable poses. We empirically set $N = 3$ for optimal performance.

4 EXPERIMENTS

We evaluate SPR in four key dimensions: **1) Overall Performance:** How does SPR compare to state-of-the-art VLA models on standard simulation benchmarks and real-robot tasks? **2) OOD Robustness:** How does SPR perform across five diverse perturbation types in LIBERO-Plus (novel backgrounds, robot initial state, language phrasings, object layouts, and lighting conditions)? **3) Ablations:** What are the individual contributions of spatial subgoal planning and the rewind mechanism? Does extended interaction time enable recovery from more complex failures? **4) Visualization:** How does SPR decompose tasks and prove error recovery effective?

4.1 Implementation Details

We initialize from MolmoAct’s mid-trained checkpoint and apply LoRA fine-tuning (rank 32, alpha 16) with action chunking following their post-training protocol. We detail the training configurations for both simulation and real-robot experiments below.

Training on Simulation (LIBERO). We evaluate two training configurations across 32 NVIDIA A100-80G GPUs. We apply the same data augmentation strategy as MolmoAct, including random cropping, resizing, and color jittering to improve robustness. Table 4 summarizes the complete training configuration.

The **separately-trained models** fine-tune both vision encoder and language model on each LIBERO subset individually with batch size 128 and learning rate 5×10^{-4} , training for 40K-80K steps until optimal validation performance is reached.

The **jointly-trained model** freezes the vision encoder and fine-tunes only the language model on all four subsets combined with a two-stage training process. First, we train on the original MolmoAct dataset for 20K steps with learning rate 1.5×10^{-4} to maintain broad manipulation capabilities. Then, we continue training on our SPR-annotated LIBERO data for 10K steps with learning rate 1.5×10^{-3} to learn progress-aware planning. During both stages, we sample data from the four subsets with a ratio of 5:5:4:8

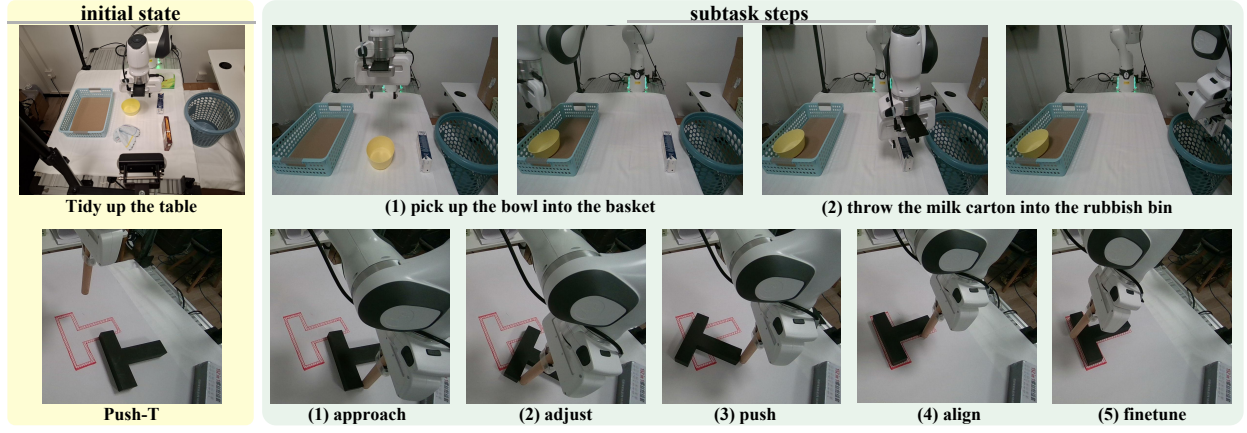


Fig. 3: Real-robot task setup and subtask decomposition for *Tidy up the Table* and *Push-T*. Each task shows the initial scene configuration alongside the model’s subtask decomposition, demonstrating that SPR produces structured and interpretable subtask plans for both pick-and-place and continuous-contact manipulation tasks.

(Spatial:Object:Goal:Long), proportional to their individual training requirements. All configurations use batch size 1024.

Training with Real-Robot Tasks. We collect 100 demonstration trajectories for *Pick up the Object* and 200 trajectories each for *Tidy up the Table* and *Push-T*. Each task is trained separately on 4 NVIDIA A100-80G GPUs with batch size 64, learning rate 5×10^{-4} , and 5,000 training steps. We freeze the vision encoder and fine-tune only the language model. The action chunk size is set to 4 during training, but only the first 2 actions are executed at each inference step to enable more responsive closed-loop control. All other hyperparameters remain consistent with the simulation setup. Table 3 summarizes the real-robot training configuration.

Inference. SPR maintains identical per-step inference cost as MolmoAct, sharing the same 7B architecture. The additional subtask planning outputs add minimal tokens compared to other predictions, resulting in negligible computational overhead. Furthermore, our error recovery mechanism operates through simple logical comparisons of subtask counts and trajectory records in the state recorder, requiring no additional model inference and thus having zero impact on execution speed. On $4 \times$ RTX 4090 GPUs, our model achieves 2.08 Hz inference speed, closely matching the baseline while enabling robust error recovery.

4.2 Environment Setup

Simulation Benchmarks. We evaluate our method on two robotic manipulation benchmarks:

- **LIBERO** [15]: A widely-used benchmark suite for language-instructed manipulation in diverse kitchen scenarios. We report results on its four distinct subtask categories: *Long* (complex multi-step tasks), *Goal* (goal-conditioned tasks), *Object* (object-centric manipulation), and *Spatial* (tasks requiring spatial reasoning).
- **LIBERO-Plus** [17]: A challenging out-of-distribution benchmark that creates over 10,000 task variants across the four LIBERO test sets through seven types of perturbations. We evaluate on five subsets: *Background* (unseen background texture), *Robot* (unseen robot initial state), *Language* (unseen instruction phrasing), *Layout* (unseen

TABLE 3: Training configuration for real-robot tasks. We collect 100 demonstrations for Pick up the Object and 200 each for Tidy up the Table and Push-T. All three tasks are trained separately with the vision encoder frozen. The action chunk size is 4 during training, but only the first 2 actions are executed at each inference step to enable more responsive closed-loop control.

Parameter	Real-Robot Tasks		
	Pick up	Tidy up	Push-T
Demonstrations	100	200	200
Steps	5K	5K	5K
Global Batch Size	64	64	64
Vision Encoder		Frozen	
GPUs (A100s)		4	
Input Images	1 Third-person + 1 Wrist-mounted		
Image Size	256×256 px		
DoF	7 (3 Translations + 3 Rotations + 1 Gripper State)		
Obs. History	No (Single-step Inputs)		
Proprioception	No		
Action Chunk	4 Steps (Predict but Only Execute 2)		

object layout), and *Light* (unseen scene lighting). Our model is trained exclusively on the original LIBERO training data, demonstrating strong generalization to these diverse unseen variations.

Real-Robot Tasks. We further validate SPR on three real-robot tasks, including one basic pick-and-place task and two challenging scenarios involving long-horizon multi-object tidying and continuous-contact pushing without grasping:

- **Pick up the Object:** Given a table with four objects (eggplant, biscuit box, toy, bowl), the robot must identify the correct object specified by the language instruction. This basic single-object task validates the model’s fundamental language grounding and manipulation capability.
- **Tidy up the Table:** As shown in Figure 3, the workspace contains 1-4 target objects (towel, biscuit box, bowl, milk carton) along with distractors such as a tissue box. The robot must organize 1-4 objects (towel, biscuit box, bowl, milk carton) on the table into their designated receptacles—bowl and towel into a basket, biscuit box and milk carton into a rubbish bin—while ignoring distractors such

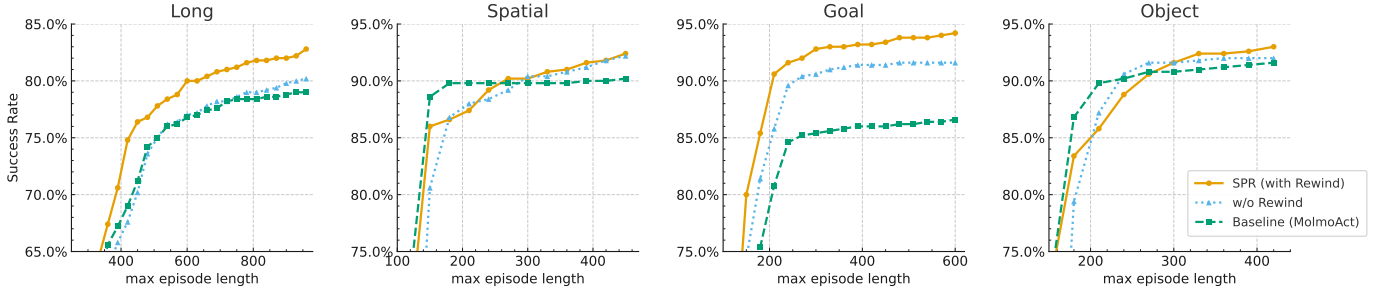


Fig. 4: Task success rate vs. maximum episode length across LIBERO subsets. Progress-aware models continue improving after the baseline plateaus, demonstrating the ability to leverage extended horizons for complex error recovery.

TABLE 4: Training configuration for LIBERO task suites. The first four columns show separately-trained models fine-tuned on individual subsets with both vision encoder and language model trainable. The last column shows the jointly-trained model with a two-stage process: first training on original MolmoAct data for 20K steps, then on our SPR-annotated data for 10K steps with data sampled at ratio 5:5:4:8 (Spatial:Object:Goal:Long).

Parameter	LIBERO Task Suite				
	Spat.	Obj.	Goal	Long	All
Steps	50K	50K	40K	80K	20K+10K
Global Batch Size	128	128	128	128	1024
Vision Frozen	No	No	No	No	Yes
GPUs (A100s)	32				
Input Images	1 Third-person + 1 Wrist-Mounted				
Image Size	256 × 256 px				
DoF	7 (3 Trans. + 3 Rot. + 1 Grip.)				
Obs. History	No (Single-step Inputs)				
Proprioception	No				
Action Chunk	8 Steps (Predict and Execute 8)				

as a tissue box. A trial is successful only when all objects are placed in their correct locations. This long-horizon setup with variable object counts serves to validate SPR’s ability to plan and execute extended subtask sequences.

- **Push-T:** As shown in Figure 3, a T-shaped block and a target zone are placed on the table. The robot uses a cylindrical end-effector to push a T-shaped block into a target zone, with success requiring full alignment. The figure illustrates how SPR decomposes this continuous-contact task into five subtask steps: approach, adjust, push, align, and fine-tune. This continuous contact manipulation task validates SPR’s generalization beyond simple pick-and-place tasks.

Evaluation Metric. We evaluate task success rate, defined as the proportion of successful task completions. For LIBERO, we test 50 episodes per task across 10 tasks in each subset. For LIBERO-Plus, we test one episode per task across over 300 tasks for each of five perturbation types. For real-robot tasks, we conduct 10 trials per task configuration.

4.3 Simulation Results

Effectiveness Evaluation on LIBERO. Table 2 presents SPR results on LIBERO. We evaluate two configurations: separately-trained models per subset (90.6%, +3.8% over MolmoAct) and jointly-trained on all subsets (91.8%, +5.0%).

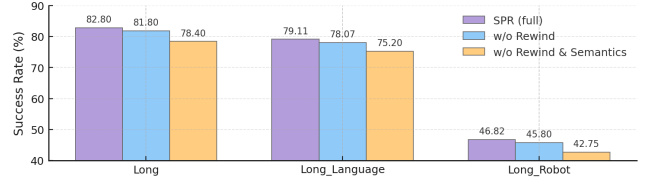


Fig. 5: Component ablation on LIBERO-Long and LIBERO-Plus variants. Performance progressively improves from w/o Rewind & Semantics (spatial coordinates only) to w/o Rewind (spatial + semantic) to Ours (full SPR with Rewind), validating the contribution of each component.

Both show strong gains on the challenging Long benchmark (+5.6% and +8.2% respectively), demonstrating that SPR effectively tackles complex multi-step manipulation. Moreover, the performance gain from joint training—rather than degradation—indicates that SPR learns generalizable progress-aware reasoning rather than overfitting to specific task distributions.

Robustness Evaluation on LIBERO-Plus. Table 1 presents results on the LIBERO-Plus benchmark under various distribution shifts. Notably, all state-of-the-art models exhibit substantial performance degradation on LIBERO-Plus compared to the original LIBERO test sets, underscoring the benchmark’s difficulty. Among all evaluated methods, SPR demonstrates the smallest overall performance drop, highlighting its exceptional out-of-distribution robustness—a direct consequence of its fine-grained, geometry-grounded planning and built-in failure recovery mechanism. Particularly noteworthy, SPR achieves the most competitive performance degradation on Language (-12.1%), Light (-5.6%), and Robot (-42.9%) perturbations. These results validate SPR’s superior capability in handling semantic ambiguity and novel robot initial configurations. The strong performance under Language shifts demonstrates the effectiveness of our semantics-based progress awareness, while the robustness to Robot configuration changes highlights the value of our Rewind mechanism in adjusting gripper poses to recoverable states. Together, these capabilities enable SPR to maintain robust performance across diverse OOD scenarios, establishing superior zero-shot adaptation compared to existing approaches.

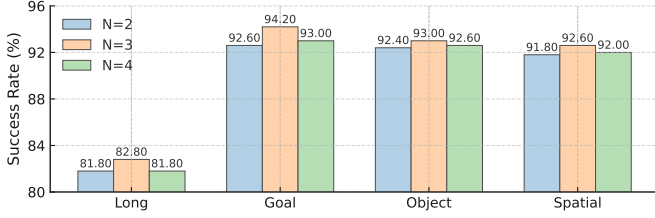


Fig. 6: Effect of rewind step count N on LIBERO. $N=3$ achieves optimal performance across all subsets.

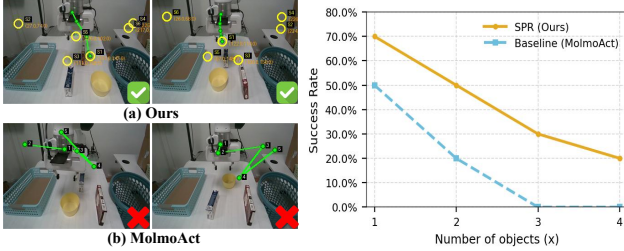


Fig. 7: Scaling analysis on real-robot *Tidy up the Table* with 1–4 objects. The planning visualization (left) compares SPR and MolmoAct on a 3-object trial, where yellow circles denote subtask waypoints and green lines indicate planned trajectories. The success rate curve (right) shows that SPR degrades gracefully as object count increases, while MolmoAct’s coarse-grained planning collapses beyond 2 objects.

4.4 Real-Robot Results

Table 5 summarizes results across three tasks. Additional setup details and visualizations for all real-robot tasks are provided in the supplementary material.

Pick up the Object. On this basic single-object task, SPR achieves 70% success rate compared to 50% for MolmoAct. Even for a single-step task, SPR’s explicit spatial grounding of the target object as a subtask waypoint enhances the model’s comprehension of the task goal and provides more precise trajectory planning toward the grasping target.

Tidy up the Table. In the 3-object setting, SPR achieves 30% success rate while MolmoAct fails entirely. SPR’s subtask planning decomposes this complex long-horizon task into clearly ordered steps, guiding the model to complete them incrementally. In contrast, the baseline’s coarse planning from the start to the final goal produces 2D waypoints that become noise rather than guidance as task complexity grows, ultimately failing to provide meaningful execution signals. A detailed analysis of how performance scales with object count is presented in Section 4.5.

Push-T. SPR achieves a 40% success rate while MolmoAct scores 0%. This task involves continuous-contact manipulation where gripper state transitions cannot delineate subtask boundaries, yet SPR successfully decomposes it into five sequential phases (approach, adjust, push, align, and fine-tune), progressively guiding the T-block toward the target zone. This confirms that SPR is not confined to pick-and-place tasks, but can perceive and track progress across diverse manipulation types through its extensible subtask planning framework.

TABLE 5: Performance on real-robot tasks. We report success rates for our method and the MolmoAct baseline across three tasks: Pick up the Object, Tidy up the Table (evaluated in the 3-object setting), and Push-T. Additional task configurations and visualizations are provided in the supplementary material. **Bold** values indicate the best results.

Method	Real-Robot Tasks		
	Pick up	Tidy up	Push-T
MolmoAct [10]	50%	0%	0%
Ours	70%	30%	40%

4.5 Ablation Study

Effect of Spatial Subgoals and Semantics. We first investigate whether subtask-level semantic planning can improve model performance—specifically, the model with only See and Plan capabilities but without Rewind. As shown in Table 6, comparing with the baseline MolmoAct model across all LIBERO test sets, the model with only See and Plan (w/o Rewind) achieves an overall 4.0% performance improvement over the baseline, strongly demonstrating the effectiveness of our progress-aware approach.

To assess the individual importance of spatial coordinates and semantic descriptions, we evaluate w/o Rewind & Semantics, which retains 2D coordinates but removes semantic generation. Figure 5 reveals that both components are essential. While w/o Rewind & Semantics outperforms the baseline on LIBERO-Long by 6%—demonstrating spatial coordinates’ contribution—it underperforms the w/o Rewind model by 3.4%. The gap also exists on LIBERO-Plus, where removing semantics significantly degrades robustness. These results demonstrate that spatial coordinates and semantic descriptions play complementary roles: coordinates provide precise spatial grounding, while semantics enable higher-level task understanding. Both are crucial for effective progress-aware reasoning.

Effect of Rewind Mechanism. We ablate the rewind mechanism to evaluate its contribution. As shown in Table 6, our full model achieves higher or comparable success rates across all LIBERO subsets compared to the model without rewind, with an overall performance gain of 1%. To further test its robustness, we evaluate on the more challenging Long suite of the LIBERO-Plus [17] benchmark, which introduces test-time variance. Results in Figure 5 show that our model with rewind maintains superior performance on both its Language and Robot variants. These findings confirm that the rewind mechanism consistently enhances performance and provides critical robustness in complex, out-of-distribution scenarios.

Impact of Inference Episode Length. Our work endows the model with progress-aware capability, granting it enhanced retry ability after failed task executions. This raises a natural question: if we allow the model more generous step limits, providing additional opportunities to complete tasks, will it achieve higher success rates? To investigate this, we evaluate three model variants on all four LIBERO subsets with a maximum episode length of 980 steps: the baseline MolmoAct model, the w/o Rewind model, and our best model. As shown in the figure 4, across all LIBERO subsets,

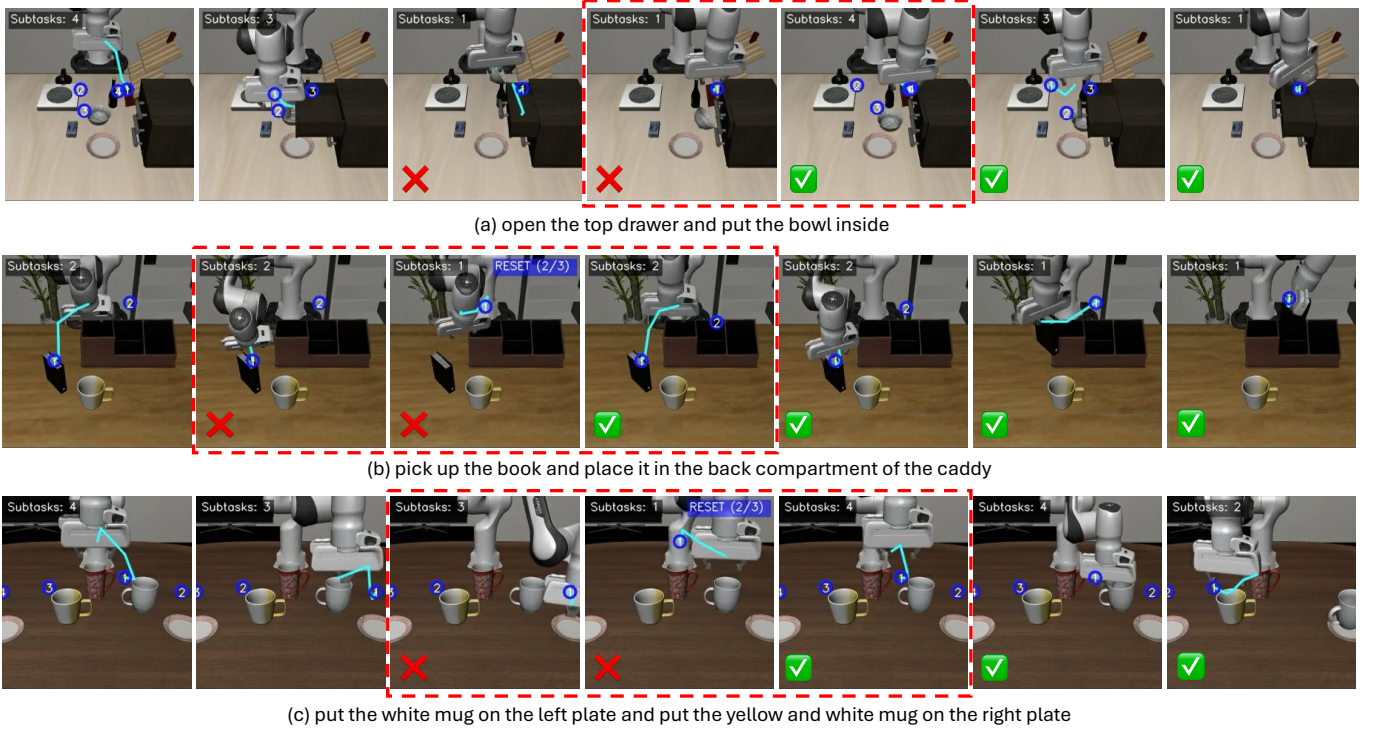


Fig. 8: SPR’s error recovery across diverse failure scenarios (red dashed boxes: error states). (a) *Dynamic replanning*: Recovers from object relocation and environment state changes by updating spatial subtasks. (b) *Robustness to suboptimal initial starts*: Mitigates challenging initial configurations by rewinding to regain spatial freedom and replanning the approach. (c) *Execution failure recovery*: Detects OOD states from failed grasps and resets to familiar configurations for retry.

TABLE 6: Ablation study on LIBERO benchmark. Both See-Plan capabilities (w/o Rewind) and the complete SPR framework (Ours) outperform the MolmoAct baseline across all subsets.

Method	Spatial	Object	Goal	Long	Avg
MolmoAct [†]	89.4%	92.4%	88.2%	72.4%	85.6%
w/o Rewind	92.6%	91.8%	92.2%	81.8%	89.6%
Ours	92.4%	93.0%	94.2%	82.8%	90.6%

[†]Results reproduced using our training setup; differs from original [10].

both our best model and the w/o Rewind model demonstrate superior ability to leverage extended episode lengths compared to the baseline, consistently achieving higher task success rates. Notably, after the baseline model’s success rate plateaus, our models continue to complete additional tasks. This validates our model’s progress-aware capability, enabling recovery from more complex error scenarios.

Furthermore, we observe that our best model exhibits faster task completion efficiency compared to the w/o Rewind model. This demonstrates that the Rewind mechanism effectively reduces time required to re-execute failed subtasks, validating the theoretical correctness of elevating the arm to provide greater operational and observational space for improved error recovery. Especially on the two more complex subsets—LIBERO-LONG and LIBERO-Goal—our best model achieves significantly faster task completion efficiency compared to both other variants.

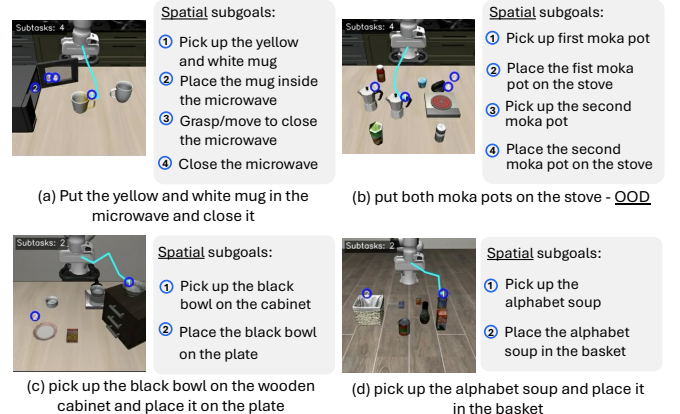


Fig. 9: Fine-grained spatial subtask decomposition across diverse manipulation tasks. (a) Complex sequential task with multiple interaction types. (b) Robust decomposition under OOD layout configurations. (c) Spatial relationship understanding with relative positioning. (d) Basic pick-and-place with clear semantic grounding. Each subtask pairs semantic descriptions with 2D spatial coordinates, enabling interpretable progress monitoring.

Effect of Rewind Step Count N . When the Rewind mechanism triggers, the model executes N consecutive “return to initial position” instructions. We investigate the optimal value by testing $N \in \{2, 3, 4\}$ across all LIBERO test sets (Figure 6). Results show $N = 3$ consistently achieves the best performance. Too few steps ($N < 3$) fail to provide suf-

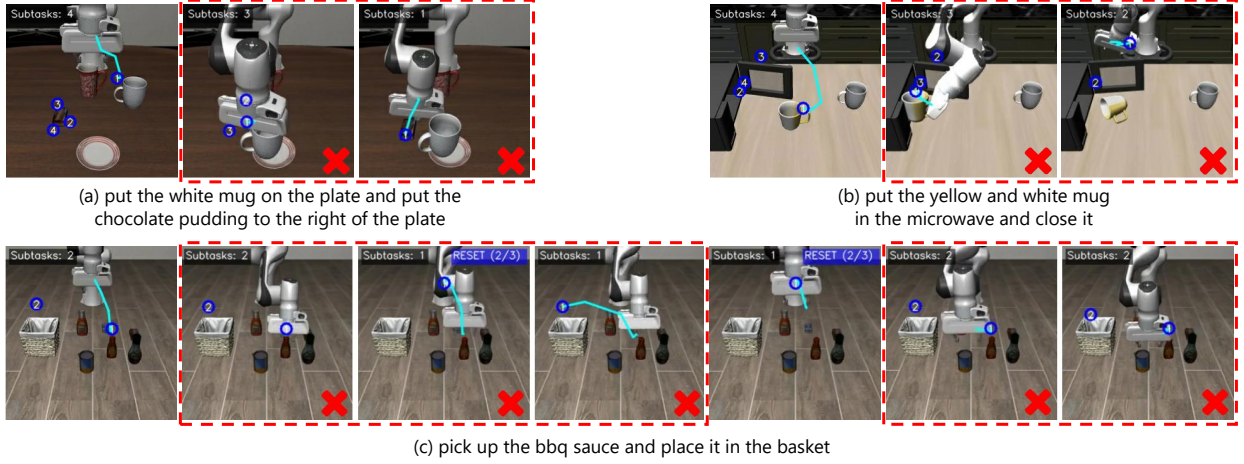


Fig. 10: Representative failure cases: (a) discrete action tokens limit precision for careful placement tasks, (b) rewind mechanism fails when robot is physically stuck, and (c) model fails to complete task despite successful rewind due to persistent spatial misalignment and action-planning inconsistency.

efficient operational space for error recovery, while too many steps ($N > 3$) cause the arm to drift excessively far from the task region. Moreover, we observe that continuous rewind instructions beyond three steps cause pose distortion, with the arm eventually moving outside the camera’s field of view. Thus, $N = 3$ balances effective error recovery with pose stability.

Scaling to Long-Horizon Real-Robot Tasks. To further investigate SPR’s advantage in extended real-world task sequences, we evaluate *Tidy up the Table* with object counts from 1 to 4. Shown in Figure 7, as the number of objects increases, both SPR and MolmoAct experience performance degradation, but SPR degrades more gracefully. When object count reaches 3 and above, MolmoAct fails entirely while SPR continues to achieve successful completions. This widening gap confirms that subtask-level progress planning becomes increasingly critical as task complexity grows, and that SPR’s structured decomposition scales effectively to challenging long-horizon scenarios.

4.6 Visualization

We present representative examples demonstrating SPR’s fine-grained spatial planning and robust error recovery across diverse scenarios.

Fine-Grained Spatial Subtask Decomposition. Figure 9 illustrates SPR’s task decomposition capabilities. (d) shows accurate decomposition of basic pick-and-place with semantic descriptions and spatial coordinates. (c) demonstrates understanding of relative positioning constraints. (b) validates robustness under distribution shift: SPR maintains accurate subtask decomposition when object layouts deviate from training configurations. (a) showcases generalization to complex sequential tasks like door closing, correctly identifying distinct manipulation phases and spatial goals.

Error Recovery through Rewind. Figure 8 (c) demonstrates handling execution failures that create unexpected state transitions. When the mug grasp fails but the arm continues to the plate position, standard VLA models struggle with this severe distribution shift—the robot reaches the target

without the required object. SPR’s progress monitoring detects the anomaly (subtask count unchanged despite reaching the goal) and triggers Rewind, returning the arm to the initial position to successfully re-grasp.

Robustness to Unexpected Perturbations. Figure 8 (a) highlights dynamic replanning under environmental perturbations. When the bowl inadvertently closes the drawer and falls to a new location, SPR’s spatial awareness enables rapid adaptation. The model relocates the bowl, updates spatial subtask coordinates, and completes the task, demonstrating spatial awareness that extends beyond in, distribution scenarios.

Handling Challenging Initial Configurations. Figure 8 (b) shows robustness to suboptimal initial poses, a key challenge in LIBERO-Plus Robot. The arm begins with poor alignment to the book, and the first grasp fails. Unlike baselines that persist with failures or knock over objects, SPR’s Rewind mechanism detects the failure and returns to the initial position, enabling a successful approach from a better angle. This autonomous recovery is valuable for deployment scenarios with variable initial conditions.

4.7 Failure Analysis

Figure 10 illustrates representative failure modes of our approach and their underlying causes. Despite the robust performance of our SPR framework, we identify three primary failure patterns that highlight remaining challenges in vision-language-action models.

Precision limitation from discrete action tokens. Our model outputs actions as discrete token sequences, which inherently limits the precision of continuous control commands. This discretization, while beneficial for leveraging language model architectures, introduces quantization errors that become particularly problematic for tasks requiring fine-grained manipulation. As shown in Figure 10(a), when tasked with placing a mug precisely at the center of a plate, the model instead places it near the edge. This occurs because the discrete action space cannot adequately capture the subtle control nuances needed for millimeter-level precision. Such failures are especially prevalent in tasks involving

careful object placement, alignment, or insertion operations where small deviations lead to task failure.

Rewind ineffectiveness when physically stuck. While our progress-aware rewind mechanism successfully detects anomalies through subtask count monitoring, it fundamentally relies on the robot’s ability to execute actions that change its state. When the robot becomes physically constrained or stuck during execution, the rewind mechanism fails because the physical obstruction prevents meaningful state transitions. Figure 10(b) demonstrates this limitation: when attempting to place a mug in a microwave, the mug becomes lodged at the microwave’s edge. Despite the model issuing rewind commands, the physical constraint prevents the robot from moving, leaving the subtask count unchanged and the anomaly detection unable to trigger proper recovery. This failure mode reveals a critical gap between progress awareness and physical state awareness—our system can detect logical inconsistencies but cannot directly sense physical impediments.

Persistent failure despite successful rewind. Perhaps most concerning are cases where our model correctly identifies execution errors, successfully triggers and executes the rewind procedure, yet remains unable to complete the task. Figure 10(c) illustrates this complex failure pattern during a grasp bbq sauce task. Initially, the model encounters spatial misalignment between the gripper and target object, correctly triggering a rewind. However, after rewinding, the model fails to correct the misalignment. In subsequent attempts, even after returning to the initial position, the model exhibits hallucination behavior with significant positional deviations from the target object. Most troublingly, even in instances where the predicted 2D trajectory and subtask waypoints accurately indicate the correct target location, the generated actions fail to align with these planned waypoints, resulting in the gripper moving to different positions than intended. This reveals a fundamental disconnect between the model’s spatial planning capabilities and its action execution—while it can correctly perceive and plan where to go, the actions it generates do not faithfully follow these plans. Such failures indicate that spatial awareness and trajectory planning alone are insufficient without ensuring consistency between planned waypoints and executed actions.

5 CONCLUSION

We introduce See, Plan, Rewind (SPR), a vision-language-action framework for robust manipulation via spatial subtask decomposition and autonomous recovery. Extensive experiments validate our approach: SPR outperforms MolmoAct by 5% on LIBERO. On the challenging LIBERO-Plus benchmark, it achieves state-of-the-art robustness, exhibiting the smallest performance degradation (−18.8% in average) across unseen instructions and initial states perturbations, underscores its superior generalization. Future work will be extending our method to diverse simulations, and real-world scenarios while handling noisy or suboptimal training demonstrations.

REFERENCES

- [1] H. Liu, A. Chen, Y. Zhu, A. Swaminathan, A. Kolobov, and C.-A. Cheng, “Interactive robot learning from verbal correction,” *arXiv preprint arXiv:2310.17555*, 2023.
- [2] L. X. Shi, Z. Hu, T. Z. Zhao, A. Sharma, K. Pertsch, J. Luo, S. Levine, and C. Finn, “Yell at your robot: Improving on-the-fly from language corrections,” *arXiv preprint arXiv:2403.12910*, 2024.
- [3] C.-P. Huang, Y.-H. Wu, M.-H. Chen, Y.-C. F. Wang, and F.-E. Yang, “Thinkact: Vision-language-action reasoning via reinforced visual latent planning,” *arXiv preprint arXiv:2507.16815*, 2025.
- [4] J. Duan, W. Pumacay, N. Kumar, Y. R. Wang, S. Tian, W. Yuan, R. Krishna, D. Fox, A. Mandelkar, and Y. Guo, “Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation,” *arXiv preprint arXiv:2410.00371*, 2024.
- [5] W. Lu, M. Ye, Z. Ye, R. Tao, S. Yang, and B. Zhao, “Robofac: A comprehensive framework for robotic failure analysis and correction,” *arXiv preprint arXiv:2505.12224*, 2025.
- [6] P. Zhi, Z. Zhang, Y. Zhao, M. Han, Z. Zhang, Z. Li, Z. Jiao, B. Jia, and S. Huang, “Closed-loop open-vocabulary mobile manipulation with gpt-4v,” *arXiv preprint arXiv:2404.10220*, 2025.
- [7] Z. Liu, A. Bahety, and S. Song, “Reflect: Summarizing robot experiences for failure explanation and correction,” *arXiv preprint arXiv:2306.15724*, 2023.
- [8] Z. Zhang, K. Zheng, Z. Chen, J. Jang, Y. Li, S. Han, C. Wang, M. Ding, D. Fox, and H. Yao, “Grape: Generalizing robot policy via preference alignment,” *arXiv preprint arXiv:2411.19309*, 2024.
- [9] M. Zawalski, W. Chen, K. Pertsch, O. Mees, C. Finn, and S. Levine, “Robotic control via embodied chain-of-thought reasoning,” *arXiv preprint arXiv:2407.08693*, 2024.
- [10] J. Lee, J. Duan, H. Fang, Y. Deng, S. Liu, B. Li, B. Fang, J. Zhang, Y. R. Wang, S. Lee, W. Han, W. Pumacay, A. Wu, R. Hendrix, K. Farley, E. VanderBilt, A. Farhadi, D. Fox, and R. Krishna, “Molmoact: Action reasoning models that can reason in space,” *arXiv preprint arXiv:2508.07917*, 2025.
- [11] R. Yang, Z. An, L. ZHou, and Y. Feng, “Seqvla: Sequential task execution for long-horizon manipulation with completion-aware vision-language-action model,” *arXiv preprint arXiv:2509.14138*, 2025.
- [12] Z. Lin, J. Duan, H. Fang, D. Fox, R. Krishna, C. Tan, and B. Wen, “Failsafe: Reasoning and recovery from failures in vision-language-action models,” *arXiv preprint arXiv:2510.01642*, 2025.
- [13] O. Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, C. Jose, V. Khalidov, M. Szafraniec, S. Yi, M. Ramamonjisoa, F. Massa, D. Haziza, L. Wehrstedt, J. Wang, T. Darce, T. Moutakanni, L. Sentana, C. Roberts, A. Vedaldi, J. Tolan, J. Brandt, C. Couprie, J. Mairal, H. Jégou, P. Labatut, and P. Bojanowski, “DINOv3,” *arXiv preprint arXiv:2508.10104*, 2025.
- [14] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, “Segment anything,” *arXiv:2304.02643*, 2023.
- [15] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.
- [16] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025.
- [17] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei *et al.*, “Libero-plus: In-depth robustness analysis of vision-language-action models,” *arXiv preprint arXiv:2510.13626*, 2025.
- [18] Q. Bu, Y. Yang, J. Cai, S. Gao, G. Ren, M. Yao, P. Luo, and H. Li, “Univla: Learning to act anywhere with task-centric latent actions,” *arXiv preprint arXiv:2505.06111*, 2025.
- [19] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, D. Salz, M. Neumann, I. Alabdulmohsin, M. Tschannen, E. Bugliarello *et al.*, “Paligemma: A versatile 3b vlm for transfer,” *arXiv preprint arXiv:2407.07726*, 2024.
- [20] S. Karamcheti, S. Nair, A. Balakrishna, P. Liang, T. Kollar, and D. Sadigh, “Prismatic vlms: Investigating the design space of visually-conditioned language models,” in *Forty-first International Conference on Machine Learning*, 2024.
- [21] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds *et al.*, “Flamingo: a visual language model for few-shot learning,” *Advances in neural information processing systems*, vol. 35, pp. 23 716–23 736, 2022.

- [22] X. Yang, J. Lu, and E. Yu, "Walking the tightrope: Disentangling beneficial and detrimental drifts in non-stationary custom-tuning," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [23] E. Yu, J. Lu, X. Yang, G. Zhang, and Z. Fang, "Learning robust spectral dynamics for temporal domain generalization," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [24] A. O'Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain *et al.*, "Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 6892–6903.
- [25] F. Ebert, Y. Yang, K. Schmeckpeper, B. Bucher, G. Georgakis, K. Daniilidis, C. Finn, and S. Levine, "Bridge data: Boosting generalization of robotic skills with cross-domain datasets," *arXiv preprint arXiv:2109.13396*, 2021.
- [26] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis *et al.*, "Droid: A large-scale in-the-wild robot manipulation dataset," *arXiv preprint arXiv:2403.12945*, 2024.
- [27] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu *et al.*, "Rt-1: Robotics transformer for real-world control at scale," *arXiv preprint arXiv:2212.06817*, 2022.
- [28] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, D. Wang *et al.*, "Spatialvla: Exploring spatial representations for visual-language-action model," *arXiv preprint arXiv:2501.15830*, 2025.
- [29] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Chormanski, T. Ding, D. Driess, A. Dubey, C. Finn *et al.*, "Rt-2: Vision-language-action models transfer web knowledge to robotic control, 2023," URL <https://arxiv.org/abs/2307.15818>, 2024.
- [30] M. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "Openvla: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.
- [31] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, " π 0: A vision-language-action flow model for general robot control. corr. abs/2410.24164, 2024. doi: 10.48550," *arXiv preprint ARXIV.2410.24164*, 2025.
- [32] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, " π 0. 5: a vision-language-action model with open-world generalization, 2025," URL <https://arxiv.org/abs/2504.16054>, vol. 1, no. 2, p. 3, 2025.
- [33] Q. Gu, Y. Ju, S. Sun, I. Gilitschenski, H. Nishimura, M. Itkina, and F. Shkurti, "Safe: Multitask failure detection for vision-language-action models," *arXiv preprint arXiv:2506.09937*, 2025.
- [34] K. Palempalli, R. Banerjee, S. Dean, and T. Bhattacharjee, "Human-in-the-loop foundation model failure recovery for robot-assisted bite acquisition," in *1st Workshop on Safely Leveraging Vision-Language Foundation Models in Robotics: Challenges and Opportunities*.
- [35] H. Chen, Y. Yao, R. Liu, C. Liu, and J. Ichnowski, "Automating robot failure recovery using vision-language models with optimized prompts," *arXiv preprint arXiv:2409.03966*, 2024.
- [36] Y. Dai, J. Lee, N. Fazeli, and J. Chai, "Racer: Rich language-guided failure recovery policies for imitation learning," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 15 657–15 664.
- [37] M. S. Sakib and Y. Sun, "Star: A foundation model-driven framework for robust task planning and failure recovery in robotic systems," *arXiv preprint arXiv:2503.06060*, 2025.
- [38] DeepSeek-AI, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.
- [39] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine, "Fast: Efficient action tokenization for vision-language-action models," *arXiv preprint arXiv:2501.09747*, 2025.
- [40] J. Cen, C. Yu, H. Yuan, Y. Jiang, S. Huang, J. Guo, X. Li, Y. Song, H. Luo, F. Wang *et al.*, "Worldvla: Towards autoregressive action world model," *arXiv preprint arXiv:2506.21539*, 2025.
- [41] C.-Y. Hung, Q. Sun, P. Hong, A. Zadeh, C. Li, U.-X. Tan, N. Majumder, and S. Poria, "Nora: A small open-sourced generalist vision language action model for embodied tasks," *arXiv preprint arXiv:2504.19854*, 2025.
- [42] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.
- [43] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu *et al.*, "Octo: An open-source generalist robot policy," *arXiv preprint arXiv:2405.12213*, 2024.